super:

Super is the keyword

It is used to access the member of super class ~~members~~ from sub class (both static & non-static members)

It can be used "only Inside Non-static context".

Eg:-

```
class A
{
  int i=10;
}
class B extends A
{
  int i=20;
}
class Driver4
{
  public static void main(String[] args)
  {
    new B();
  }
}
```

```
{
  System.out.println("Overloading Object");
  System.out.println(i);
  System.out.println(super.i);
}
```

O/P:

```
Overloading Object
20
10
```

Eg2: class Accounts

```
{
 int acno;
 String name;
 double bal;
 String ifsc;
 Accounts () { }
 Accounts (int acno, String name, double bal, String ifsc)
 {
  this.acno = acno;
  this.name = name;
  this.bal= bal;
  this.ifsc= ifsc;
 }
 public void displayDetails ()
 {
 System.out.println ("Account number :" + acno);
 System.out.println ("Account name:" + name);
 System.out.println ("Account balance :" + bal);
 System.out.println ("IFSC code of branch :" + ifsc);
 System.out.println ("=========");
 }
}

class LA extends Accounts.
{
 double loanamt;
 double emi;
 LA() { }
 LA(int acno, String name, double bal, String ifsc, double
                                  loanamt, String ifsc)
 {
   super (acno, name, bal, ifsc);
```

```
        this.loanamt = loanamt;
        this.emi = emi;
    }
    public void displayDetails()
    {
        super.displayDetails(); // access superclass members.
        System.out.println("The loan amount: " + loanamt);
        System.out.println("Emi for the loan: " + emi);
        System.out.println("=========================");
    }
}
class Driver3
{
    public static void main(String[] args)
    {
        Accounts a1 = new LA(101, "Smith", 10000, "ICICI001",
                             200000, 10000);
        a1.displayDetails(); // Method overriding.
    }
}
```

a1 ←



O/P:

```
Account number: 101
Account name: Smith
Account balance: 10000
IFSC code of branch: ICICI001
==========
The loan amount: 200000
Emi for the loan: 10000
==========
```

28/02

final:

* It is a keyword
* It is a non-access Modifier which can be prefixed to four members:-
    1) Class
    2) Variables
    3) Methods

final Variable:

* A Variable which is prefixed with final modifier is known final Variable
* A final Variable behaves like a constant (value once assigned cannot be modified).
* In java, we have following final variable based on the place of declaration
    1) Local final variable
    2) Global final Variable

Local final Variable:

A local variable which is prefixed with final modifier is known as Local final Variable

A local variable which is a final member will have same characteristics as that of Normal local variable. [Refer characteristics of Local variable]. exeept we cannot re-assign it different value.

Eg: class P6
{
public static void main (String [] args)
{
final int I = 10;
System.out.println (I);
I = 20; // CTE → cannot reassign final variable I
}
}

final Global variable:

A Global variable [static & Non-static] which is prefixed with final Modifier is known as Global final variable.

A final Global variable must be declared & initialized with the help of initializers. Else we get CTE

Eg: class G2.
{
final static int I; // CTE
public static void main (String [] args)
{

* A static final variable must be initialized with the help of any one of the static initializers as shown below:

Eg: class G4
{
public final static int I;
static
{
I = 20;
}
public static void main (String [] args)
{
System.out.println (I); //20
}
}

} OR final static int I = 20;

* A non-static final variable must be initialized with the help of any one of the non-static initializers.

NOTE: CONVENTION: The final variables must always be declared in upper case only.

final Method:

A method which is declared by prefixing with final modifier is known as a final method

(8)

NOTE1 :) A final method gets inherited.

```
class P7.
{
 final public void test().
  {
   System.out.println("From test()");
  }
}P:
class P8 extends P7
 {
  public static void main(String[] args)
   {
    P8 obj= new P8();
    obj.test();
   }
 }
```

O/P :
From test()

2) A final method cannot be Overridden.

```
class Addition
 {
  final public void add(int a, int b)
   {
    int res = a-b;
    system.out.println(res);
   }
 }
class Add extends Addition ①
 {
  public void add(int a; int b)
   {
    int res = a+b;
    System.out.println(res);
   }
```

O/P :
CTE
add(int, int,) from Add cannot override add(int, int) in Addition.

▶ final class:

A class which is prefixed with final modifie is known as final class.

NOTE: A final class cannot be inherited.

Eg:
```
final class P9
{
  public static void main (String[] args)
  {
    System.out.println("Hello world!");
  }
}
class P10 extends P9
{
}
```

o/p:

error: class P9 cannot be inherited.